

Project 13 — Speaking Script (English, full deck)

Target ≈ 18–19 min. Casual student tone — *say it, don't read it*.

Speaker split (Emir's): 1–3 **Emir** · 4–8 **Dima** · 9–12 **Emir** · 13–15 **Dima** · 16–17 **Emir** · 18–31 **Ilya**

Handoffs (say the next name — keeps it smooth): Emir → **Dima** (sl. 4) → Emir (sl. 9) → **Dima** (sl. 13) → Emir (sl. 16) → **Ilya** (sl. 18).

 **Guard the budget — two danger zones:** 1. **Slides 18–24 (loss curves) = ONE montage**, ~30 s total, click fast. Two sentences for all seven — do *not* explain each. 2. **Slide 16 (3 bugs)** — say the short version, don't re-derive each fix.

 **Two speaker conflicts to confirm with the team** (deck notes vs. this split): - **Sl. 18–24** are written in Dima's voice in the deck; here they're Ilya's (neutralised wording). - **Sl. 30 (Takeaways)** is marked [EMIR] in the deck and reads like Emir's close — here it's under Ilya per the split. **Probably still Emir — decide.**

 **Numbers — use the deck's, not the cheatsheet's:** WER **14.4 %** (epoch 2, best) vs ceiling **5.75 %**, on **655** held-out clips; MCD **10.5** → **6.0 dB**. (The cheatsheet's "29.2 %" is a stale 20-clip number — don't quote it.)

Slide 1 · Title — [EMIR]

Hi — we're Dima, Emir and Ilya, and this is Project 13, Text to Speech. We took a model that already speaks perfectly well... and taught it to talk like a Star Wars battle droid. On purpose. *(beat)* Let me tell you why.

Slide 2 · The goal — [EMIR]

Just running a pretrained TTS model isn't really a project — it already produces clean speech. The actual goal is to **change the voice itself**: fine-tune the model to speak as a **B1 battle droid**. And we picked a droid on purpose — a subtle voice makes a demo ambiguous, but a droid you hear instantly. You can literally **hear that the fine-tuning worked**, before we show a single metric. *(beat)* Although we did the metrics too — it's a deep learning course.

Slide 3 · Project architecture — [EMIR]

One word on how three of us worked on one model without stepping on each other. I set the repo up **contract-first**: data, model and evaluation are separate packages that only talk through small fixed interfaces in `core` — a manifest and one result object — never each other's internals. So the model stays **swappable**, and we could all build in **parallel**. One Makefile runs the whole pipeline, the same commands locally and on Colab, and there's an offline mock plus a smoke test, so the whole thing wires up in seconds with no GPU. With that skeleton in place — **Dima** takes the model.

Slide 4 · Three ways to change a TTS voice — [DIMA]

Thanks. There are three ways to change a voice. You can **pre-process** the input — good for an accent, but limited. You can **post-process** the output audio — I tried it, it sounds fake and robotic. Or you **bake** the voice into the model itself by fine-tuning. That's the real way, and that's what we did — the droid becomes the model's own voice.

Slide 5 · Where to start: the ideas — [DIMA]

We brainstormed four directions: a **B1 droid voice** — a timbre change you hear instantly; a **heavy accent** — phoneme-level; **emotions** — prosody; and **stuttering** — rhythm. Emotions and stuttering were too shallow, so we dropped them. We kept the droid as the core and left the accent as future work.

Slide 6 · Our choice: bake in a droid voice — [DIMA]

So: bake a B1 droid voice into the model. One consistent voice across the whole dataset, an effect you can hear without any metric, and — the fun part — it lets me play with the training itself. Since post-processing sounded bad, I built the training data with **RVC** instead.

Slide 7 · Building the dataset — RVC — [DIMA]

But I need data — a lot of paired text and audio in the droid voice. You can't record 24 hours of a droid. *(beat)* I checked. So I built it: I took all of LJSpeech and ran it through **RVC** — voice conversion. Think of it as a **Photoshop for voices**: it repaints the timbre to the droid but keeps the words, so the original transcripts still match. **13,100 paired clips, for free** — published on the Hugging Face Hub, QR's on the slide.

Slide 8 · The trade-off we chose — [DIMA]

One honest trade-off. Because the data comes from RVC, the model learns from **RVC's output** — so its quality **can't beat that teacher**; RVC is the upper bound. But in return, the model produces the droid voice **itself**, end-to-end, with **no second model in production**. Post-processing would mean running RVC every single time; baking means the voice is built in. A bit less ceiling, full autonomy — and Ilya actually measured where that ceiling sits. Back to **Emir** for the model.

Slide 9 · Why VITS — [EMIR]

Thanks, Dima. We chose VITS for three reasons. **One — it's end-to-end**: text straight to waveform in one model. Traditional TTS is like a **composer** who writes the notes and a separate **musician** who plays them — two models you have to train and keep in sync. VITS is the **one-person band**: text in, sound out, nothing to sync. **Two — phoneme-level access**, which we wanted for the Russian-accent idea later. **Three — a strong pretrained checkpoint** to start from, `vits-ljs`. Under the hood it's a conditional VAE with a normalizing flow, a GAN decoder, and a stochastic duration predictor.

Slide 10 · The generator: why it isn't enough — [EMIR]

Here's the generator. In training it gets **two** inputs — the text and the real audio — and it learns to rebuild the waveform, comparing them on the **mel-spectrogram**, which is basically a picture of the sound: an equalizer drawn out over time. That comparison is an **L1 loss** — the average pixel-by-pixel difference between our mel and the real one. And on its own L1 rewards *safe, averaged* audio, so the result comes out blurry and muffled — it just can't capture fine detail. We actually **heard** this on a test run: the generator alone gave a weak result, the droid timbre didn't come through. That's the gap a **discriminator** fills. One thing to stress — the real audio is used in **training only**; at inference there's no audio, it's purely **text** → **speech**.

💬 *Backup — if asked "why L1 and not L2?":* it's the standard VITS / HiFi-GAN recipe, and we kept it on purpose. L2 squares the errors, so it over-penalizes outliers and over-smooths — even blurrier audio; L1 treats errors proportionally, keeps more detail, robust to outliers. Even L1 blurs, though — the real fix is the discriminator, not the choice of norm.

Slide 11 · Discriminator: built from scratch — [EMIR]

And here's the catch: **Hugging Face ships VITS inference-only** — there's no discriminator in the package. So we **wrote one from scratch**, HiFi-GAN style. The idea is a **counterfeiter and a**

detective: the generator prints the audio, the discriminator tries to spot the fake, and under that pressure the generator gets convincingly real. There are two — a **multi-period** discriminator that folds the waveform by prime periods to catch pitch and harmonics, and a **multi-scale** one over three time scales. They also expose their feature maps, which gives us a **feature-matching loss**. Without this, the decoder never learns realistic audio — it's the **missing half** of the VITS objective.

Slide 12 · Lining up text & audio: MAS — [EMIR]

Last model piece: alignment. We have maybe ten text tokens but hundreds of audio frames, and nothing tells us which frame belongs to which token. **Monotonic Alignment Search** is a dynamic-programming search — Viterbi-like — that finds the best **monotonic** mapping from text to frames. Monotonic means speech moves left to right, can't skip or repeat — so it **can't babble**. As a bonus, it tells us how long each token should last, which is the target for the duration loss. **Dima** — how'd training go?

Slide 13 · How training works — [DIMA]

One training step: encode the text to a prior, encode the real audio to a latent, run the flow, use **MAS** to line up text with audio frames, slice a short segment, decode it to a waveform, compute the losses. Then a standard GAN alternation: the **discriminator** trains on real versus detached fake, then the **generator** trains on **five losses** to fool it — reconstruction weighted **45**. The messy plumbing — two optimizers, detach, clipping, mixed precision — I wrapped in decorators, so the step itself reads clean.

Slide 14 · Abstraction — [DIMA]

Quick thing I'm proud of. The GAN loop is messy — two optimizers, detach, backward, clipping, the AMP scaler, accumulation — all tangled around two tiny core steps. I moved all that wiring into **decorators** and left only the real step logic in the methods. Read the method, the algorithm is linear; need the plumbing, read the decorator. Same idea as `torch.nn.Module` hiding backward — the complexity doesn't vanish, it just goes where it belongs.

Slide 15 · Where & how we trained — [DIMA]

On compute: Colab first — and it fought me, sessions dropped around 15 minutes, disk filled with checkpoints. (*beat*) Very generous. So I moved to my local **RTX 5060, 8 gigs**. To fit 8 gigs:

batch size 1 + gradient accumulation, mixed-precision **fp16**, and zero dataloader workers, because the Windows loader kept crashing. With that working — **Emir**, the bugs.

Slide 16 · Problems & fixes: nothing converged — [EMIR] (*deck notes say Dima*)

At first, **nothing converged**. Reconstruction was stuck, the duration loss was in the hundreds, and the voice came out way too fast. Three bugs. **One** — the duration loss was ~100× too large because it was averaged wrong; fixing the normalization dropped it from three hundred to under two. **Two** — gradient clipping was so tight it killed the generator's updates; loosening it let reconstruction finally drop. **Three** — a fresh, untrained discriminator was corrupting the pretrained decoder, so we **warmed the discriminator up alone for 1500 steps** and lowered the learning rate. After that — real training: reconstruction fell from 0.85 to 0.40, and the rhythm became correct. The lesson: **a falling loss does not prove good speech** — only listening, and later WER, caught these.

Slide 17 · Training setup & cost — [EMIR] (*deck notes say Dima*)

The numbers. The final clean run was **5 epochs, ~1.9 h each — roughly 9.7 GPU-hours total** — on the local RTX 5060. Key config: **learning rate 1e-4**, much gentler than from-scratch; batch size 1 with grad accumulation; mixed precision; reconstruction weighted 45. But the honest number is bigger: about **3× that in failed runs** and debugging, plus baking the whole 13,100-clip dataset through RVC. The 9.7 hours is only the final clean pass — the real work was everything before it. **Ilya**, over to the results.

Slides 18–24 · Loss curves — [ILYA] **MONTAGE** — *click fast, ~30 s total (deck notes say Dima)*

Slide 18 · Reconstruction (L1 mel)

Quick montage of the training losses — short version, everything's stable. Reconstruction drops sharply then plateaus around 0.4; the voice is learned early, which is why we don't take the last checkpoint.

Slide 19 · Duration — (*click*)

Duration sits at a healthy 1.6 after the normalization fix — correct rhythm.

Slide 20 · KL divergence — (click)

KL settles around 1.7 — the text side learned to match the audio, alignment works.

Slide 21 · Adversarial (generator) — (click)

Adversarial loss oscillates around 4 without running away.

Slide 22 · Discriminator — (click)

The discriminator we built stays stable around 5.5 — the warm-up kept it from overpowering the decoder.

Slide 23 · Feature-matching — (click)

Feature-matching spikes during warm-up, then settles around 2.

Slide 24 · Generator total — (land here)

And total generator loss, around 28, flat and stable. But — and this is the point — **a converging loss does not prove the speech is understandable**. That's exactly what evaluation checks.

Slide 25 · Evaluation: what we measure & why — [ILYA]

Three numbers, all **lower-is-better**. **WER and CER** from Whisper measure intelligibility — can a listener understand it — and they work for any voice, droid included. **MCD** measures spectral closeness to the **B1 droid reference** — that's the key one, it tells us whether we learned the timbre. And we measure the **ceiling**: Whisper on the **real B1 recordings** scores **5.75 %** word error rate, so no model trained on this data can beat that. The B1 twist that matters: we score MCD against the **droid** reference, not the original female voice — otherwise we'd just be measuring how *un*-droid-like we are.

Slide 26 · How we evaluate — [ILYA]

How, quickly. We hold out the **last 5 %** of the data, which the model never saw. Before-and-after is a single **checkpoint swap** on the same prompts — apples-to-apples. The eval is a contract-first module: feed it a manifest of texts and reference audio, it returns one result object with WER,

CER and MCD in one pass. And the ASR is **Whisper — frozen, off-the-shelf, never part of training** — so there's no circularity.

Slide 27 · Results: vs the ceiling — [ILYA]

The payoff, on the full **655-clip** held-out set. Two things. **First, timbre:** MCD — the spectral distance to the droid — drops from **10.5 to 6.0 dB**. Hard proof the fine-tuning worked; the voice moved to the droid. **Second, intelligibility against the ceiling:** our best checkpoint, epoch 2, is **14.4 % WER**, versus the **5.75 %** of the real B1 recordings. It can't beat its teacher — but it lands close. So: we gained the new voice, at a small, **measured** cost.

Slide 28 · Demo — [ILYA]

Okay — the part you actually came for. Same sentence, before... (*play base*) ...and after. (*play fine-tuned*) Same words, completely different voice. (*beat*) ...Roger, roger.

Slide 29 · Failure modes & limitations — [ILYA]

Honest limitations. **One** — intelligibility sits above the ceiling, ~14 vs 5.75 % — understandable, but less clean than the base. **Two** — it's not a hard sci-fi robot: RVC moves a speaker's **timbre**, not the ring-mod buzz; that mechanical buzz would be a separate DSP effect. **Three** — GAN artifacts early, and it over-fits past the best epoch, which is why we don't take the last checkpoint. And the **Russian accent** is still future work.

Slide 30 · Takeaways & future work — [ILYA per split / ⚠ deck = EMIR — probably Emir's close]

To wrap up: we **built and published our own droid dataset** with RVC, **fine-tuned VITS** onto that voice, and **re-implemented the whole GAN training stack from scratch** — discriminator, losses, the loop — because the library ships inference-only. The result is a clear, measured trade-off: the timbre moved to the droid, at a small intelligibility cost, **bounded by the data's own ceiling**. The next step to close that gap: **freeze the text/timing front-end** during fine-tuning, which keeps durations stable — plus finally the Russian accent.

Slide 31 · Thank you — [ALL]

That's us — Dima on the model, Ilya on evaluation, Emir on data and infra. Thanks for listening — questions? (*Route by ownership: model → Dima · evaluation → Ilya · data & engineering → Emir. Repeat each question before answering.*)

Q&A quick-fire (repeat the question first, then route)

- **"Why is WER higher than the base?"** → it's the trade-off; the ceiling is 5.75 % and we're near it; freezing the front-end is the fix. (*Ilya*)
- **"Couldn't you just use RVC and skip the model?"** → RVC needs input audio, it can't go from text. We used it only as a teacher to build the data, then baked the voice into one text-to-speech model — no RVC at runtime. (*Dima/Emir*)
- **"Is it really a droid or just a different person?"** → RVC transfers a speaker's timbre; the hard mechanical buzz is a separate DSP step, out of scope. (*Dima*)
- **"You wrote the discriminator yourselves?"** → yes — HF ships VITS inference-only; we wrote MPD + MSD + feature-matching, without it the decoder never learns realistic audio. (*Emir*)
- **"Why L1 and not L2?"** → standard VITS / HiFi-GAN recipe; L2 over-penalizes outliers and over-smooths, L1 keeps more detail and is robust; even L1 blurs, so the real fix is the discriminator. (*Emir*)
- **"Can you reproduce the numbers?"** → yes, `make eval` ; full held-out 5 % (655 clips), Whisper frozen. (*Ilya*)
- **"Why VITS over Tacotron2 / SpeechT5?"** → end-to-end, one model, phoneme access, strong checkpoint. (*Emir*)
- **"How did three people not collide?"** → contract-first, mocks + smoke test, one Makefile. (*Emir*)